08_Greedy

1. 核心思想 | The Core Idea

贪婪算法的逻辑非常直观：**每一步都采取局部最优解**（The locally optimal choice），并希望通过这些局部最优的选择，最终达到**全局最优解**（The globally optimal choice）。

- **特点**：简单、快。
- **代价**：并不总能得到“最完美”的答案，但在很多复杂问题中，它能给出一个“足够好”的近似值。

---

2. 经典案例：电台集合覆盖问题 | The Set Covering Problem

假设你正在办一个电台节目，你想让全加拿大的听众都能听到。每个电台覆盖的省份不同，有的重叠。你想用**最少的电台**覆盖**所有的省份**。

- **完美主义（穷举法）**：计算出所有电台的组合。如果你有 n 个电台，组合数就是 2^n。如果有 100 个电台，计算量比宇宙寿命还长。
- **贪婪逻辑（Greedy Strategy）**：
  1. 选一个覆盖了最多“尚未覆盖省份”的电台。
  2. 重复第一步，直到所有省份都被覆盖。

> **结果**：贪婪算法算得极快，它的复杂度是 $O(n^2)$。虽然它选出的电台组合可能比最完美的方案多 1-2 个，但比起算一亿年，这个代价完全可以接受。

---

3. NP 完全问题 | NP-Complete Problems

这是 CS 领域的终极挑战之一。有些问题（如**旅行商问题 TSP** 或上述的**集合覆盖问题**），目前还没有能在合理时间内算出完美答案的算法。

- **识别特征**：
  - 涉及“所有组合”的问题。
  - 不能拆分成小问题（Dijkstra 那种可以拆分的就不是 NP 完全）。
  - 随着元素增加，计算量爆炸式增长。
- **对策**：面对这种问题，别死磕完美答案，**直接上贪婪算法**。

---

4. 贪婪算法 vs. 动态规划 | Greedy vs. Dynamic Programming

维度	贪婪算法 (Greedy)	动态规划 (Dynamic Programming)
选择逻辑	**局部最优**：走一步看一步。	**全局考量**：考虑所有子问题的关系。
准确性	可能不是最优解（近似解）。	绝对的最优解。
计算速度	极快。	较慢。
适用场景	NP 完全问题、实时决策。	背包问题、最长公共子序列。

---

5. 你的 AI 视角 | The AI Perspective

在机器学习里，贪婪思想无处不在。比如**决策树（Decision Tree）**在分叉时，通常就是用贪婪算法选出当前最好的特征。你未来学 AI，会发现我们经常在“精度”和“算力成本”之间做这种取舍。

💡 思考题 (Sijia's Challenge)

假设你正在 Augustana 的食堂排队打饭，你的盘子空间有限（体积 V）。这里有：

- **龙虾**：价值 100，体积 80
- **牛排**：价值 60，体积 50
- **汉堡**：价值 50，体积 40

如果你是个贪婪的人，你会**先拿**哪一个？

如果你想让盘子里的总价值**最高**，你会怎么选？

（提示：贪婪算法会让你先拿最贵的，但也许拿两个便宜的反而总分更高。这就是贪婪算法的局限。）

这章的概念比较轻松，你觉得这种“不求完美只求快”的思维，是不是很符合你现在多拿几个 Offer（U of A, MBZUAI）做备选的逻辑？